

Lab test, MATH 3311, Term 2, 2024

Q1) For Python users: Import the following:

```
from numpy import pi, sin, exp
from numpy.linalg import cond, det
from scipy.linalg import norm, qr
```

Let n be a positive integer and consider the $(n+1) \times (n+1)$ matrix

$$A = [a_{i,j}]_{1 \leq i,j \leq n+1} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 1 & \frac{1}{3} & \frac{1}{3^2} & \frac{1}{3^3} & \frac{1}{3^4} & \cdots & \frac{1}{3^n} \\ 1 & \frac{1}{5} & \frac{1}{5^2} & \frac{1}{5^3} & \frac{1}{5^4} & \cdots & \frac{1}{5^n} \\ 1 & \frac{1}{7} & \frac{1}{7^2} & \frac{1}{7^3} & \frac{1}{7^4} & \cdots & \frac{1}{7^n} \\ \vdots & \vdots & \vdots & \vdots & & \vdots & \\ 1 & \frac{1}{(2n+1)} & \frac{1}{(2n+1)^2} & \frac{1}{(2n+1)^3} & \frac{1}{(2n+1)^4} & \cdots & \frac{1}{(2n+1)^n} \end{pmatrix}$$

Fix $n=11$.

(a) Generate the matrix A efficiently. Compute the sum of all entries of the matrix A and store the value in the variable **sumA**.

(b) Calculate the condition number of A in 2-norm using the Matlab/Python command 'cond' and store it in the variable **condA**.

(c) Is A ill-conditioned? Write a comment in the variable 'comment1' to answer this question, i.e., type **comment1 = 'yes'** if A is ill-conditioned or **comment1 = 'no'** if not.

(d) Compute the determinant of A and store the value in the variable **detA**.

(e) Is A nonsingular? Write a comment in the variable 'comment2' to answer this question, i.e., type **comment2 = 'yes'** if A is nonsingular or **comment2 = 'no'** if A is singular.

(f) From the QR-decomposition, $A=QR$ where Q is an orthogonal matrix and R is an upper triangular matrix. Find the first five entries in the diagonal of Q . Store the values in the row vector **diag5Q**.

In the next part n is not 11.

(g) The polynomial $P(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4$ interpolates the function $f(x) = \frac{\sin(10\pi x)}{100e^x}$ at the points $0, 1/9, 1/7, 1/5, 1/3$. Use the matrix A for an appropriate choice of n to compute the unknown coefficients a_0, a_1, a_2, a_3, a_4 and store the values in the column vector **veca**.

Q2) For Python users: Import the following:

```
from numpy import zeros, exp
from scipy.optimize import fsolve
```

The Newton method can be used to estimate a root of a given function g , i.e. a value x^* such that $g(x^*) \approx 0$. It is defined by the iteration

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)}, \quad n = 0, 1, 2, 3, \dots \quad (1)$$

Let $f(x) = (x - 1)^4 + \exp(x)$, for $x \in [0, 3]$.

The aim is to estimate the inverse of this function at the point 2, $f^{-1}(2)$, using Newton's method.

- Define a variable **x0** and set it to 3. Use this value as starting value for the Newton iteration.
- Modify Newton's iteration (1) so that it can be used to approximate the inverse of a function. Write a Matlab/Python program which uses this modified Newton iteration (do Not use build-in functions like fzero in Matlab or fsolve in Python to do that). Compute x_1 , x_2 , and x_3 , and store the results in **x1**, **x2**, and **x3**, respectively.
- Run the Newton iteration till the difference between successive approximations is smaller 10^{-8} , that is, $|x_{n+1} - x_n| < 10^{-8}$. Store the result in **xe**.
- Find $f^{-1}(2)$. Store the result in **invf2**.

Q3) For Python users: Import the following:

```
from numpy import ones, sqrt, pi, sin, linspace
from scipy.special import gamma
```

Part A

In this part, the goal is to approximate the integral $I := \int_0^1 (2 - 2x^2)^{1.5} \sqrt{x} dx$ using a composite trapezoidal rule. Recall that the composite trapezoidal rule for an integrand g is given by $\int_a^b g(x) dx \approx h[g(x_0)/2 + g(x_1) + g(x_2) + \dots + g(x_{N-2}) + g(x_{N-1}) + g(x_N)/2]$, where $a = x_0 < x_1 < x_2 < \dots < x_{N-1} < x_N = b$ are equally spaced points in the interval $[a, b]$ and $h = \frac{b-a}{N}$, i.e. $x_n = a + nh$.

- Set $N = 9$ and store it in the variable **N** and define a variable **h** given by $h = \frac{b-a}{N}$.
- Compute an approximation to the integral I using the composite trapezoidal rule with $N + 1$ points in the following way:
 - Define a row vector **x** of $N + 1$ equally spaced points whose first component is 0 and whose last component is 1
 - Define a row vector **w** given by $[1, 2, 2, \dots, 2, 2, 1]$ of length $N+1$.
 - Compute the function $f(x) = (2 - 2x^2)^{1.5} \sqrt{x}$ at the points $x_0, x_1, \dots, x_N$, and store your results in the row vector variable **f**.

2.4) Store the value of this approximation in the variable **trapf**.

3) Use the true solution $\frac{16}{15\pi^{0.5}} \left(\Gamma(0.75) \right)^2$ to compute the absolute errors from trapezoidal rule.

Store the absolute error in **errtrap**. Note: $\Gamma(0.75)$ is the standard gamma function of 0.75 . Hint: `gamma(0.75)` computes $\Gamma(0.75)$ in both Matlab and Python.

Part B

In this part, the goal is to extend the composite trapezoidal rule (defined above) to approximate the two dimensional integral $J := \int_0^1 \int_0^1 \sin(xy) \sqrt{x+2y} dx dy$. As in **Part A**, set

$N = 9$, $h = (b - a)/N$, $x_n = a + nh$, and $y_m = a + mh$ for $0 \leq n, m \leq N$. Then, with

$F(x, y) = \sin(xy) \sqrt{x+2y}$, the integral J can be approximated by 2D composite trapezoidal rule

$trapF = \frac{h^2}{4} \mathbf{w} \mathbf{F} \mathbf{w}^T$ where the $(N + 1)$ by $(N + 1)$ matrix $\mathbf{F} = [F_{i,j}]$ with $F_{i,j} = F(x_i, y_j)$ for

$0 \leq i, j \leq N$. Compute $trapF$ and store the value of this approximation in the variable **trap2dF**.

Q4) **For Python users:** Import the following:

```
from numpy import ones, linspace, loadtxt, pi, exp, cos, sin
from scipy.interpolate import CubicSpline
from scipy.fft import fft
from scipy.linalg import lstsq
```

The goal of this question is to approximate the crude average annual oil price data by using cubic spline, linear least square and trigonometric polynomial approximations.

The first column in the data set below contains the numbers from 1 to 10 and the second column contains annual average crude oil price data in USD over a period of years.

1	77.64
2	94.53
3	68.17
4	39.68
5	56.99
6	65.23
7	50.8
8	43.29
9	48.66
10	93.17

a) Store the first column in the column vector **tdata** and the second column in the column vector **ydata**.

b) Compute the not-a-knot cubic spline which interpolates the data. Evaluate the spline at $teval = [1.3, 2.3, 3.3, \dots, 9.3]$ and store the results in the row vector **yspteval**.

c) Consider the linear least-squares fit to the given data of the form $y(t) = x_1 + x_2 t + x_3 \exp(t) + x_4 \cos(\pi t/2)$. Compute the coefficient vector $x = (x_1, x_2, x_3, x_4)$ and store the results in the column vector **xLS**. Matlab students should use \backslash command '\', while Python students should use lstsq to compute the vector x .

d) Evaluate the least squares approximation at teval and store the results in the row vector **yLSteval**.

Let w (with entries w_i for $1 \leq i \leq m$) be the complex DFT of the data **ydata**, which can be calculated using the Matlab\Python function fft.

e) The number of data points $m = 10$. Calculate $a_0 = w_1/m$ and store the result in **a0**.

f) Calculate $a_k = \frac{2}{m} \Re w_{k+1}$ for $k = 1, 2, 3, 4$, and $a_5 = \frac{w_6}{m}$, where $\Re w_{k+1}$ is the real part of w_{k+1} ,

and store the results in the row vector **a**. Hint: For Matlab users, you need to use the command 'real(x)' to find the real part of x . For Python users, you need to use the command 'x.real' to find the real part of x .

g) Given that the row vector $b = [b_k] = [3.9119, 1.7992, -0.4331, -4.9655, 0]$, calculate the DFT approximation

$$a_0 + \sum_{i=1}^5 a_i \cos(2\pi i teval / m) + b_i \sin(2\pi i teval / m)$$

and store the results in the row vector **ydfteval**.

Q5) **For Python users:** Import the following:

```
from numpy.linalg import solve, norm
from numpy import diag, exp, ones, zeros, linspace, inf
```

In this question we use a finite difference scheme to approximate the solution to the following

boundary value problem: $-\frac{d^2 u}{dx^2}(x) + 3 \exp(-3x) \frac{du}{dx}(x) + 9u(x) = 0$ for $0 < x < 1$, with

$u(0) = 0$ and $u(1) = 1$.

Part A

a) Set $N = 5$ and $h = 1/5$.

b) Define the points $x_n = nh$, for $n = 0, 1, \dots, N$, and save them in the row vector called **x**.

c) Approximate $u(x_n)$ by U_n using the finite difference scheme

$$-\frac{U_{n+1} - 2U_n + U_{n-1}}{h^2} + 3 \exp(-3x_n) \frac{U_{n+1} - U_{n-1}}{2h} + 9U_n = 0 \quad \text{for } 1 \leq n \leq N-1,$$

with $U_0 = 0$ and $U_N = 1$, to obtain a system of linear equations

$\left(-\frac{1}{h^2}A + \frac{3}{2h}B + 9C\right)W = R$ where A , B and C are $(N-1) \times (N-1)$ matrices and R is a

column vector of length $N-1$, given by

$$R = \left(0, 0, 0, \frac{1}{h^2} - \frac{3}{2h} \exp(-3(N-1)h)\right)^T.$$

Store the matrix A in the variable **A**, the sum of all entries of the matrix B in the variable **sumB**, the matrix C in the variable **C**, and the right-hand side R in the column vector **R**.

d) Solve the system of linear equations for W and store the results in the column vector **Uapprox**, given by the values $[U_1; U_2; U_3; U_4]$.

Part B

Be careful not to change the values of the variables **x**, **Uapprox**, **A**, **sumB**, **C**, **R** from **Part A**.

e) The true solution of the given boundary-value problem is $u(x) = \frac{1 - \exp(3x)}{1 - \exp(3)}$. Define a row vector **u** of length $N-1$ with entries $u(x_1), u(x_2), \dots, u(x_{N-1})$. Compute the infinity norm error between the true vector solution **u** and the approximate vector solution **U** for $N = 40$ and $N = 80$. Store your error results in the row vector **E** = $[E_1, E_2]$ respectively.